hcwave – Local Docker Setup

Component-by-component guide to running the Hospital AI Wave 1 platform on a laptop with Docker Compose

Repository: hcwave (branch master)
Compose project name: `hcwave`
Document date: 30 October 2026
Source of truth: `docker-compose.yml`, `.env.example`, `services/backend/Dockerfile`, `services/staff-ui/Dockerfile`, `config/`

Contents

1. Purpose and what “local” means
2. Architecture of the local stack
3. Prerequisites

4. Quick start
5. Component reference (5.1 shared definition, 5.2 postgres, 5.3 redis, 5.4 migrate, 5.5 api, 5.6 worker, 5.7 staff-ui, 5.8 volumes, 5.9 networking, 5.10 start-up order)
6. Docker images in detail
7. Configuration system
8. Offline substitutes for Azure services
9. Using the platform locally (including scheduled jobs)
10. Day-to-day operations
11. Running the tests
12. Switching a component to a real service
13. Troubleshooting
14. Security notes for local use
15. Appendix – environment variable reference

1. Purpose and what “local” means

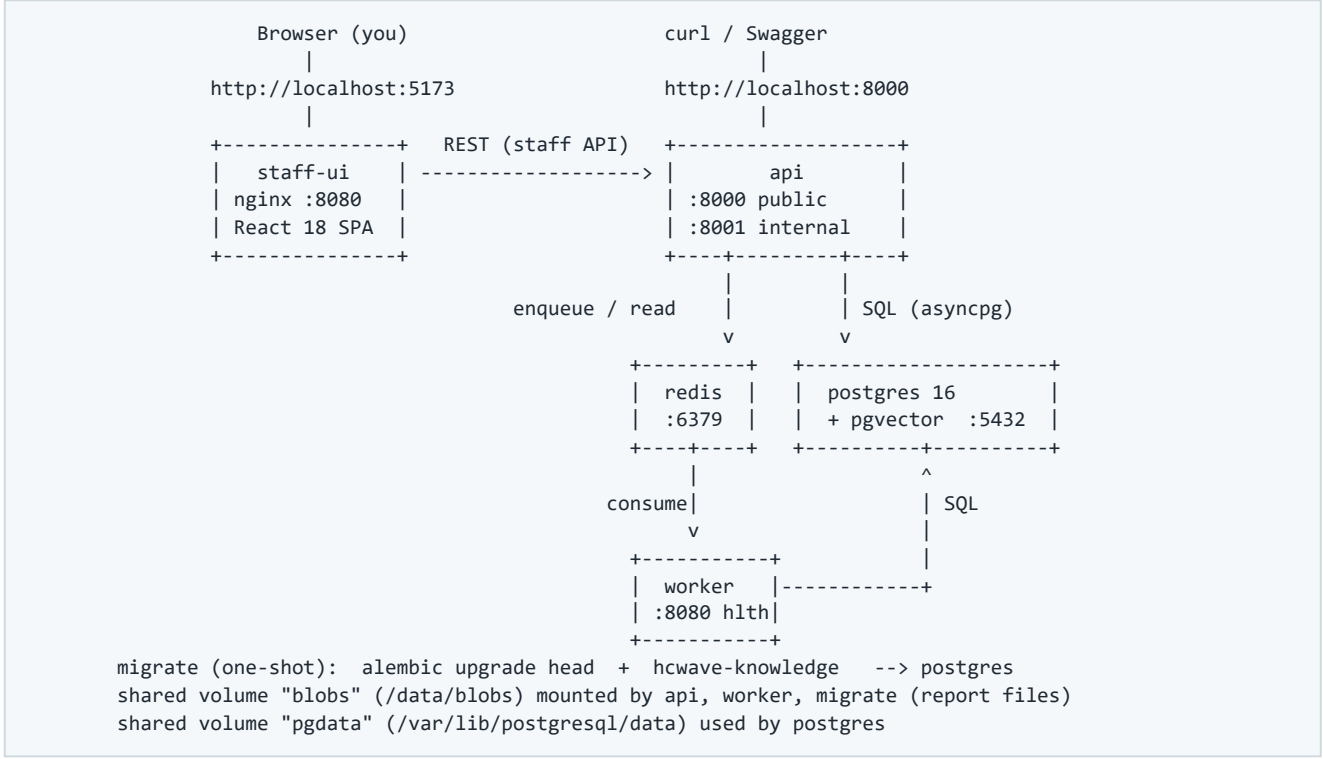
The hcwave repository delivers five hospital patient-engagement products (WhatsApp assistant, reminders, report delivery, feedback and reputation, management dashboard) as one platform. In production it runs on Azure (AKS, Service Bus, Azure OpenAI, ACS WhatsApp and so on). The Docker Compose stack described here runs **the same application code** entirely on a laptop with **no cloud account and no internet dependency** after the images are built.

This works because the code depends on *ports* (interfaces), and each Azure service has an offline *adapter* selected by one value in `.env`. Locally the WhatsApp channel prints to the console, the language model is a deterministic fake, files go to a Docker volume and the queue is Redis.

> **Not for production.** Authentication is disabled, passwords are defaults and developer-only routes are on. Use the Helm chart and Bicep (see the main as-built document) for any shared or cloud environment.

2. Architecture of the local stack

Six Compose services are defined: five long-running (postgres, redis, api, worker, staff-ui) and one one-shot (`migrate`).



Message flow in the local stack

1. A patient message arrives. In real life that is a WhatsApp webhook; locally you call `POST /dev/simulate` or `POST /webhooks/console` on the **api**.
2. The api normalises the event, resolves the hospital from `channel_id` and either handles it directly (dev simulate) or places it on the `inbound-messages` queue in **redis**.
3. The **worker** consumes the queue, locks the conversation row, runs the LangGraph assistant (emergency check → intent → flow or FAQ) and reads/writes **postgres**.
4. Replies go through the messaging adapter. Locally the `console` provider records and logs them instead of calling WhatsApp.
5. Staff open the **staff-ui**, which calls the api's `/staff/*` endpoints for inbox, takeover, tickets, appointments and dashboard.

3. Prerequisites

Item	Requirement
Docker	Docker Desktop (Windows/macOS) or Docker Engine + Compose v2 plugin (Linux). Verify with `docker --version` and `docker compose version`. Compose v2 is required (the file uses `name:`, YAML anchors and `depends_on` conditions).
Windows	Docker Desktop with the WSL 2 backend. Make sure Docker Desktop is **running** (whale icon steady) before any command; otherwise you get "failed to connect to the docker API".
Resources	4 CPU cores and 6–8 GB RAM allotted to Docker are comfortable. Disk: roughly 3–4 GB for images and build cache.
Free host ports	5432 (Postgres), 6379 (Redis), 8000 (API), 8001 (internal API), 5173 (staff UI). Stop any local Postgres/Redis using 5432/6379 or change the left-hand side of the port mapping.
Internet	Needed only for the first build (base images from Docker Hub, Python packages from PyPI, npm packages from the npm registry).
Optional	`make` (Git Bash/WSL on Windows) for the shortcuts `make up/down/logs/simulate`; `curl`; Python 3.12 and Node 20 only if you run tests or the UI outside Docker.

4. Quick start

```
cd hcwave                       # repository root (where docker-compose.yml lives)
cp .env.example .env            # PowerShell: Copy-Item .env.example .env
docker compose up --build       # first build takes several minutes
# in a second terminal:
curl -XPOST localhost:8000/dev/simulate -H "content-type: application/json" -d "{\"text\":\"hi\"}"
```

Then open:

URL	What you see
http://localhost:5173	Staff app (Inbox, Tickets, Front desk, Dashboard). Sign-in is disabled; you are “Dev Admin”.
http://localhost:8000/docs	Interactive Swagger UI for the API (enabled by `server.docs_enabled` in `dev.yaml`).
http://localhost:8000/healthz	Liveness – `{"status":"ok"}`.
http://localhost:8000/readyz	Readiness – `{"status":"ready"}` once the database answers, otherwise HTTP 503.

Equivalent Makefile shortcuts: `make up` (creates `.env` if missing, builds, starts detached), `make logs`, `make simulate`, `make down`.

> **PowerShell quoting.** Escape the JSON quotes as shown above, or use `curl.exe` with a file: `curl.exe -XPOST localhost:8000/dev/simulate -H "content-type: application/json" -d "@msg.json"`. Alternatively call the endpoint from the Swagger UI.

5. Component reference

All services come from `docker-compose.yml`. The two application services and the migration job share one definition through a YAML anchor (`x-backend: &backend`), which is why they have identical environment and volumes.

5.1 Shared backend definition (`x-backend`)

Setting	Value and meaning
build	Context = repository root, Dockerfile `services/backend/Dockerfile` (see section 6).
image	`hcwave/backend:local` – all three backend services reuse the one image tag, so it is built once.
env_file	`.env` – every variable from your copy of `.env.example` is injected.
environment (overrides)	`APP_ENV=dev` (selects `config/environments/dev.yaml`); `DATABASE_URL=postgresql+asyncpg://hcwave:hcwave@postgres:5432/hcwave` (host `postgres` is the Compose service name); `DATABASE_SSL_MODE=disable`; `QUEUE_PROVIDER=redis`; `REDIS_URL=redis://redis:6379/0`; `LOCAL_STORAGE_ROOT=/data/blobs`.
volumes	`blobs:/data/blobs` – named volume where the local storage adapter keeps report files.
depends_on	`postgres` must be *healthy*; `redis` must be started.

> Values in `environment:` win over `env_file`. That is why `.env` may say `QUEUE_PROVIDER=in_memory` and `localhost` URLs (for running outside Docker) while the containers still use Redis and the Compose hostnames.

5.2 postgres – system of record

Property	Detail
Image	`pgvector/pgvector:pg16` – PostgreSQL 16 with the `pgvector` extension available.
Credentials	User `hcwave`, password `hcwave`, database `hcwave` (local only).
Port	5432 published to the host, so tools such as psql, DBeaver or pgAdmin can connect: `postgresql://hcwave:hcwave@localhost:5432/hcwave`.
Storage	Named volume `pgdata` at `/var/lib/postgresql/data`. Data survives `docker compose down`; it is erased by `down -v`.
Health check	`pg_isready -U hcwave` every 5 s, up to 20 retries. Dependent services wait for "healthy".
What it stores	Created by Alembic migrations 0001 and 0002: hospitals, departments, doctors, doctor_schedules, schedule_exceptions, patients, patient_contacts, consent_events, conversations, messages, appointments, reports, otp_challenges, report_access_logs, feedback, tickets, notification_logs, staff_users, audit_logs, knowledge_chunks, sync_states, plus reporting views.
Row-level security	Enabled (`database.row_level_security: true`); each request/session is scoped to a hospital so tenants cannot read each other's rows.
pgvector	Used only when `KNOWLEDGE_PROVIDER=pgvector`. The default local provider is `keyword`, which needs no embeddings.

5.3 redis – message queue

Property	Detail
Image	`redis:7-alpine`.
Port	6379 published (useful with `redis-cli` or RedisInsight).
Role	Replaces Azure Service Bus. Queues defined in `config/app.yaml`: `inbound-messages`, `delivery-status`, `report-events`. The api produces, the worker consumes, so the two run as separate containers.
Persistence	None configured (no volume). Queued, unprocessed messages are lost if the container is removed – acceptable for development.
Why not in_memory?	The `in_memory` provider only works when API and worker share one process. Compose runs them separately, so Redis is mandatory here.

5.4 migrate – one-shot initialiser

Property	Detail
Image	`hcwave/backend:local` (same image, different command).
Command	`sh -c "alembic -c /app/alembic.ini upgrade head && hcwave-knowledge"`
Step 1 – Alembic	Applies `migrations/versions/0001_initial_schema.py` (core tables, constraints, indexes) and `0002_security_and_reporting.py` (row-level security policies, reporting views, grants). Idempotent: re-running does nothing if already at head.
Step 2 – hcwave-knowledge	Syncs hospital YAML into the `hospitals` table and indexes FAQ/policy content (`config/knowledge/demo-hospital/faq.yaml`) into `knowledge_chunks` via the configured knowledge adapter.
restart	`"no"` – it runs once and exits with code 0.
Consumers	`api` and `worker` declare `depends_on: migrate: condition: service_completed_successfully`, so they start only after the schema exists. If migrate fails, they never start – read `docker compose logs migrate`.

5.5 api – FastAPI service

Property	Detail
Command	`hcwave-api` (entry point `hcwave.api.serve:main`).
Ports	**8000** public listener (webhooks, report portal, staff API, docs, health). **8001** internal listener (job triggers for schedulers; private-only in production, published locally for convenience).
Process model	Two uvicorn servers in one asyncio process. With `in_memory` queue the worker consumers would also run in this process; with Redis they do not.
Start-up work	Loads layered config, builds the dependency container (DB pool, adapters), syncs hospital YAML to the database.

Route groups

Prefix	Purpose	Local auth
`/healthz`, `/readyz`	Liveness and database readiness.	none

`/webhooks/{provider}`, `/webhooks/eventgrid`	Inbound patient messages and delivery receipts (`console`, `meta_cloud`, `acs_whatsapp` payloads). Verified by signature in cloud setups.	none for console
`/dev/simulate`	Injects a fake patient message, processes it synchronously and returns the bot's replies. Present only when `server.dev_routes_enabled` is true.	none
`/staff/*`	Used by the staff UI: `/me`, `/conversations` (+ messages, takeover, reply, release), `/tickets` (list, patch), `/appointments`, `/consent`, `/dashboard/summary`, `/dashboard/embed`.	disabled (`AUTH_PROVIDER=disabled` → Dev Admin)
`/r/{hospital}/{token}`	Patient report portal: landing page, OTP request, verify (OTP or date of birth), view via short-lived signed URL.	token + OTP/DOB
`/internal/*` (port 8001)	`POST /internal/jobs/{name}`, feedback request/status, consent. Called by Logic Apps / Functions in Azure.	shared secret `INTERNAL_API_TOKEN`
`/docs`, `/openapi.json`	Swagger UI and OpenAPI schema.	none (dev only)

5.6 worker – queue consumer

Property	Detail
Command	`hcwave-worker` (entry point `hcwave.worker.main:main`).
Role	Consumes `inbound-messages`, `delivery-status` and `report-events` concurrently. Runs the LangGraph assistant: emergency keyword check → language detection → intent → deterministic flows (booking, manage, availability, feedback) or FAQ answer from the knowledge store → handover when confidence is low.
Ordering guarantee	A per-conversation row lock keeps each patient's messages in order even with several workers.
Health	Small FastAPI app on port 8080 inside the container (`/healthz`, reports unhealthy if no consumer loop ran for 120 s). Not published to the host.
Scaling	`docker compose up --scale worker=3` works because Redis is shared.

5.7 staff-ui – React app behind nginx

Property	Detail
Build	Context `services/staff-ui`; two-stage Dockerfile (Node 20 builds with Vite, unprivileged nginx serves static files). Image `hcwave/staff-ui:local`.
Port	Host **5173** → container **8080**.
Runtime environment	`AUTH_MODE=none` (no Entra sign-in); `API_BASE_URL=http://localhost:8000` (the browser, not the container, calls the API); `CSP_CONNECT_SRC=http://localhost:8000` (adds the API to the Content-Security-Policy `connect-src` so the browser is allowed to call it).
Runtime config	At start-up `docker/40-runtime-config.sh` writes `/tmp/config.js` from the environment (apiBaseUrl, authMode, clientId, tenantId, apiScope, pollSeconds). nginx serves it as `/config.js` with `no-store`, so one image works for every environment.
nginx hardening	Non-root (UID 101), security headers (CSP, X-Frame-Options DENY, nosniff, HSTS, no-referrer), immutable caching for `/assets/`, SPA fallback to `index.html`, `/healthz` endpoint.

Pages	**Inbox** (conversations, takeover, reply, hand back; emergencies first), **Tickets** (service-recovery tickets, status updates), **Front desk** (appointments, consent capture), **Dashboard** (KPIs; Power BI/Grafana embed when configured).
Polling	Refreshes every 5 s by default (`POLL_SECONDS`).
depends_on	`api` (start order only).

> If the browser console shows CORS or CSP errors, check that `STAFF_UI_ORIGIN` in `.env` equals the URL you use for the UI (`http://localhost:5173`) and that `API_BASE_URL` / `CSP_CONNECT_SRC` equal the API URL you reach from the browser.

5.8 Volumes

Volume	Mounted at	Contents / lifecycle
`pgdata`	postgres: `/var/lib/postgresql/data`	All database files. Removed only by `docker compose down -v` or `docker volume rm hcwave_pgdata` .
`blobs`	api, worker, migrate: `/data/blobs`	Local object store used by `STORAGE_PROVIDER=local` ; holds report files that the portal serves. Shared so the worker can write and the api can serve.

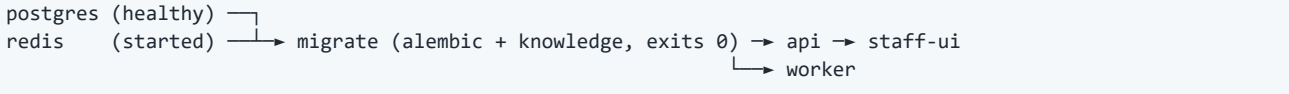
Compose prefixes volume names with the project name: `hcwave_pgdata` , `hcwave_blobs` .

5.9 Networking and ports

Compose creates one bridge network (`hcwave_default`). Services reach each other by service name (`postgres` , `redis` , `api`). Only the following are published to the host:

Host port	Service : container port	Purpose
5432	postgres : 5432	Database tools
6379	redis : 6379	Queue inspection
8000	api : 8000	Public API, docs, webhooks
8001	api : 8001	Internal job API
5173	staff-ui : 8080	Staff web app

5.10 Start-up order and health checks



Typical first-run timeline: images build (minutes) → postgres becomes healthy (~5–15 s) → migrate runs (~5–20 s) → api and worker start → staff-ui starts. Confirm with `docker compose ps` ; `migrate` shows “Exited (0)”, all others “running”.

6. Docker images in detail

6.1 Backend image (`services/backend/Dockerfile`)

One image serves three roles, chosen by the command: `hcwave-api`, `hcwave-worker` and `hcwave-jobs <job>` (used as Kubernetes CronJobs in other environments).

Stage / feature	Detail
Build stage	Base `python:3.12-slim` (override with `--build-arg PYTHON_IMAGE=…` for a private mirror). Creates a virtualenv in `/opt/venv` and installs the package with extras `EXTRAS=azure,portable` (Azure SDKs plus Redis, boto3 and OTLP exporter).
Corporate proxy	Optional BuildKit secret: `docker build --secret id=ca_bundle,src=ca.pem …` sets `PIP_CERT` for TLS-inspecting proxies.
Runtime stage	Fresh slim image; copies only `/opt/venv`, `config/`, `alembic.ini` and `migrations/`. Tests are excluded via `.dockerignore`.
Environment	`HCWAVE_CONFIG_DIR=/app/config`, `HCWAVE_ENV_FILE=/nonexistent/.env` (the container never reads a .env file; Compose injects variables), `APP_ENV=prod` by default (Compose overrides to `dev`), unbuffered Python output.
User	Non-root UID/GID 10001.
Ports	EXPOSE 8000, 8001, 8080.
Health check	Python one-liner calling `http://127.0.0.1:8000/healthz` every 30 s (relevant for api; the worker and migrate containers override semantics via their own commands).

6.2 Staff UI image (`services/staff-ui/Dockerfile`)

Stage	Detail
build	`node:20-alpine`; `npm ci` then `npm run build` (TypeScript check + Vite bundle to `dist/`).
runtime	`nginxinc/nginx-unprivileged:1.27-alpine`; copies `dist`, the nginx template and the runtime-config script; user 101; port 8080. `NGINX_ENVSUBST_FILTER=CSP_` limits template substitution to the CSP variables.

6.3 Build tips

```
docker compose build                 # build all images
docker compose build --no-cache api  # force a clean backend build
docker compose build --progress=plain # show full build output when diagnosing
docker images | findstr hcwave       # PowerShell; use grep on Linux/macOS
```

The `.dockerignore` excludes `.git`, virtualenvs, caches, `node_modules`, `.env*` (except the example), `docs`, `infra`, `deploy`, `workflows` and backend tests, which keeps the build context small and prevents secrets from entering an image.

7. Configuration system

7.1 Layers

1. `config/app.yaml` – base settings and all provider sub-sections.
2. `config/environments/${APP_ENV}.yaml` – overrides. Locally `dev.yaml`: console messaging, fake LLMs, keyword knowledge, local storage, `auth.provider: disabled`, `dev_routes_enabled: true`, `docs_enabled: true`, scheduler engine none.
3. Optional overlay directory `HCWAVE_CONFIG_OVERLAY_DIR` (used by Helm in Kubernetes).

Any value may reference environment variables as `${VAR}` or `${VAR:-default}`. Secrets therefore never appear in YAML.

7.2 Configuration files you may edit

Path	Purpose
`config/hospitals/demo-hospital.yaml`	Tenant definition: name, timezone, languages (ml, en, hi), address, feature switches, contacts, emergency message and keywords per language, knowledge files. Copy the file to onboard another hospital – no code change.
`config/knowledge/demo-hospital/faq.yaml`	FAQ and policy documents indexed for the assistant. After editing, re-run the indexer (section 9).
`config/messaging/templates.yaml`	WhatsApp template definitions used for anything the platform initiates (reminders, reports, OTPs, feedback, alerts).
`config/locales/{en,hi,ml}.yaml`	All bot strings per language.
`config/prompts/*.yaml`	Prompts: intent classifier, FAQ answer, feedback tagger.
`config/environments/*.yaml`	dev, test, staging, prod overrides.

> `config/` is **copied into the image** at build time. After editing YAML run `docker compose up --build -d` (or `build` then `up -d`) so containers see the change. To avoid rebuilding while iterating, add a bind mount such as `- ./config:/app/config:ro` to the `x-backend` volumes in a `docker-compose.override.yml`.

8. Offline substitutes for Azure services

This table is the key to understanding what each local component stands in for. Production values are shown for context; they are selected by the same variables.

Capability	Variable	Local value	Azure / production value and local behaviour
WhatsApp channel	MESSAGING_PROVIDER	console	`acs_whatsapp` (Azure Communication Services) or `meta_cloud`. Console adapter records outbound messages in memory and logs them; `/dev/simulate` returns them.
Queue	QUEUE_PROVIDER	redis (Compose) / in_memory (single process)	`azure_service_bus` fed by Event Grid.
LLMs (4 profiles)	LLM_INTENT / REPLY / TAGGER / EMBEDDING _PROVIDER	fake	`azure_openai`, `anthropic_foundry`, `openai_compatible`. The fake model gives deterministic, keyword-based intent and canned answers – no API cost, no network.
Knowledge search	KNOWLEDGE_PROVIDER	keyword	`azure_ai_search` or `pgvector` (needs a real embedding provider).
Language detection	LANGUAGE_PROVIDER	heuristic	`azure_translator`. Heuristic detects Malayalam/Devanagari by Unicode range.
Content safety	SAFETY_PROVIDER	none	`azure_content_safety` (guardrails and prompt shields).
File storage	STORAGE_PROVIDER	local	`azure_blob` (user-delegation SAS) or `s3`. Local writes under `/data/blobs`.
Email	EMAIL_PROVIDER	console	`acs_email` or `smtp`. Console records the email instead of sending it.
SMS / OTP fallback	SMS_PROVIDER	console	`acs_sms` or `msg91`. Console records the SMS and logs only the purpose and recipient (never the text), so OTP codes are not visible; verify with date of birth on the portal locally.
Hospital system (HIS)	HIS_PROVIDER	csv	`rest_api` or `fhir`.
Dashboard	DASHBOARD_PROVIDER	builtin	`power_bi` or `grafana`.
Staff authentication	AUTH_PROVIDER	disabled	`entra_id`.
Internal API auth	INTERNAL_AUTH_MODE	shared_secret	`entra_id`. Local token: `dev-internal-token` (`INTERNAL_API_TOKEN`).
Database	DATABASE_URL	Compose postgres	PostgreSQL Flexible Server with Entra token auth (`DATABASE_AUTH_MODE=azure_ad`) and SSL required.
Scheduler	SCHEDULER_ENGINE	none	`logic_apps` or `k8s_cronjob`. Locally jobs are run by hand (section 9).
Telemetry	TELEMETRY_EXPORTER	none	`azure_monitor` or `otlp`.

Identity	AZURE_IDENTITY_MODE	default	Workload identity in AKS. Not used when all providers are offline.

9. Using the platform locally

9.1 Chat as a patient

```
curl -XPOST localhost:8000/dev/simulate -H "content-type: application/json" -d "{\"text\":\"hi\"}"
```

The response lists the bot's replies, each with `kind` (text, buttons, list), `text`, `choices` (id + title) and the template name when a template was used. Continue the conversation by sending one of the returned ids as `reply_id`:

```
curl -XPOST localhost:8000/dev/simulate -H "content-type: application/json" -d "
{\"reply_id\":\"menu:book\"}"
```

The request body fields are `from_phone` (default +919812345678), `text`, `reply_id` and `channel_id` (default `console`, which maps to the demo hospital through `DEMO_HOSPITAL_CHANNEL_IDS`). Use a different `from_phone` to act as a different patient.

9.2 Things to try

Scenario	How
Book / reschedule / cancel	Follow the menu ids through `/dev/simulate`; the appointment appears in the staff UI Front desk page.
Emergency path	Send text "chest pain" (or the Malayalam/Hindi keyword). The fixed emergency message is returned before any AI step.
FAQ answer	Ask about OPD timings or parking; the answer is taken from `faq.yaml`.
Human handover	Send an unclear request or a medical question; open Inbox in the staff UI, take over, reply, and hand back.
Consent	Send "STOP": the patient is opted out immediately; "START" opts back in.
Staff API	Use Swagger at `/docs` → `GET /staff/conversations`, `/staff/tickets`, `/staff/dashboard/summary`.

9.3 Scheduled jobs (run manually)

No scheduler runs locally. Trigger jobs through the CLI in a one-off container, or the internal API:

```
# CLI (uses the same image, network and volumes)
docker compose run --rm api hcwave-jobs reminders
docker compose run --rm api hcwave-jobs reminders --rules day_before,prep --hospital demo-hospital
docker compose run --rm api hcwave-jobs briefing

# Internal API (port 8001, shared-secret header per your auth config)
curl -XPOST localhost:8001/internal/jobs/reminders -H "Authorization: Bearer dev-internal-token"
```

Job	What it does
reminders	24-hour, 2–3-hour, missed-appointment and follow-up reminders; sent once per appointment (idempotent via `notification_logs`).
feedback	Sends feedback requests 2–4 hours after OPD / one day after discharge.
tickets-escalate	Escalates service-recovery tickets that miss their SLA.

briefing	Builds and sends the five-line daily management briefing.
his-doctors, his-visits, his-reports	Pull doctors, visits and reports from the HIS connector (CSV locally).
maintenance	Housekeeping such as expiring OTPs and tokens.
knowledge-reindex	Rebuilds the knowledge index from the YAML content.

Re-index FAQ content after editing it: `docker compose run --rm api hcwave-knowledge --hospital demo-hospital`.

10. Day-to-day operations

Task	Command
Start in background	`docker compose up --build -d`
Status	`docker compose ps`
Follow logs (all / selected)	`docker compose logs -f` · `docker compose logs -f api worker`
Stop, keep data	`docker compose down`
Stop and **delete data**	`docker compose down -v`
Restart one service	`docker compose restart worker`
Rebuild one service	`docker compose up --build -d api`
Shell in a container	`docker compose exec api sh` (the image is slim; no bash guaranteed)
Open psql	`docker compose exec postgres psql -U hcwave -d hcwave`
Inspect Redis queues	`docker compose exec redis redis-cli keys "*"`
Re-run migrations	`docker compose run --rm migrate`
Check current migration	`docker compose run --rm api alembic -c /app/alembic.ini current`
Scale workers	`docker compose up -d --scale worker=3`
Full reset (fresh DB)	`docker compose down -v && docker compose up --build`
Disk clean-up	`docker system prune` (careful: removes all unused Docker data)

Useful SQL checks:

```
select id, name from hospitals;
select count(*) from knowledge_chunks;
select id, state, updated_at from conversations order by updated_at desc limit 5;
select direction, left(body,60) from messages order by created_at desc limit 10;
```

11. Running the tests

The backend has 48 automated tests that need a real PostgreSQL. Tests are excluded from the image, so run them from the host (Python 3.12) against the Compose database:

```
docker compose up -d postgres
# create a separate test database once
docker compose exec postgres psql -U hcwave -c "create database hcwave_test"
cd services/backend
pip install -e ".[dev,azure,portable]"
set TEST_DATABASE_URL=postgresql+asyncpg://hcwave:hcwave@localhost:5432/hcwave_test   # PowerShell:
$env:TEST_DATABASE_URL="..."
pytest -q
ruff check src tests migrations
ruff format --check src tests migrations
```

Test modules cover the API, assistant flows, configuration loading, feedback, messaging adapters, reminders, reports and security. Staff UI: `cd services/staff-ui && npm ci && npm run build`.

12. Switching a component to a real service

Edit `.env` and recreate containers (`docker compose up -d`). Examples:

Goal	Set in .env
Real Azure OpenAI replies	`LLM_INTENT_PROVIDER=azure_openai`, `LLM_REPLY_PROVIDER=azure_openai`, `LLM_TAGGER_PROVIDER=azure_openai`, `AZURE_OPENAI_ENDPOINT=…`, `AZURE_OPENAI_API_KEY=…` (or sign in with `az login` and leave the key empty, mounting credentials). Match `LLM_*_MODEL` to your deployment names.
Any OpenAI-compatible server (e.g. Ollama)	`LLM_REPLY_PROVIDER=openai_compatible`, `OPENAI_COMPAT_BASE_URL=http://host.docker.internal:11434/v1`
Semantic search with pgvector	`KNOWLEDGE_PROVIDER=pgvector` and a real `LLM_EMBEDDING_PROVIDER`; then re-index.
Real email	`EMAIL_PROVIDER=smtp`, `SMTP_HOST`, `SMTP_USERNAME`, `SMTP_PASSWORD`, `EMAIL_SENDER`. A local catcher such as MailHog works well.
Real WhatsApp (Meta Cloud API)	`MESSAGING_PROVIDER=meta_cloud`, `META_ACCESS_TOKEN`, `META_APP_SECRET`, `META_VERIFY_TOKEN`, a public HTTPS tunnel (ngrok / dev tunnels) to `/webhooks/meta_cloud`, and `DEMO_HOSPITAL_CHANNEL_IDS=<phone number id>`.
Staff sign-in with Entra ID	`AUTH_PROVIDER=entra_id`, `STAFF_API_AUDIENCE`; for the UI set `AUTH_MODE=msal`, `ENTRA_CLIENT_ID`, `ENTRA_TENANT_ID`, `STAFF_API_SCOPE` on the staff-ui service.
Production-like safety	Set `DEV_ROUTES_ENABLED=false` to remove `/dev/simulate`.

> Never commit `.env`. It is git-ignored and excluded from images. If you put real keys in it, treat the laptop as holding secrets.

13. Troubleshooting

Symptom	Cause and fix
“failed to connect to the docker API … dockerDesktopLinuxEngine”	Docker Desktop is not running. Start it, wait until it reports “Engine running”, retry.
“port is already allocated” / bind error	Another program uses 5432, 6379, 8000, 8001 or 5173. Stop it or change the host side of the mapping, e.g. `"15432:5432"`.
`env file .env not found`	Run `cp .env.example .env` first.
api/worker never start, stay “created”	`migrate` failed. `docker compose logs migrate`. Common: Postgres not ready (retry), YAML error in config, missing `APP_SECRET_KEY`.
Config error mentioning an undefined variable	A `${VAR}` without default is unset. `APP_SECRET_KEY` and `DATABASE_URL` have no default; both are provided by .env/Compose.
Build fails fetching packages (SSL / proxy)	Behind a TLS-inspecting proxy, build with `--secret id=ca_bundle,src=corp-ca.pem` or set a mirror via `PYTHON_IMAGE` / `NODE_IMAGE` build args.
Staff UI loads but lists are empty / “network error”	API not reachable from the browser, CORS or CSP mismatch. Verify `http://localhost:8000/healthz` and the UI origin settings (see 5.7). Check `http://localhost:5173/config.js`.
`/dev/simulate` returns 404	Dev routes disabled: `APP_ENV` is not `dev` or `DEV_ROUTES_ENABLED=false`.
Replies are empty or the same canned text	Expected with the fake LLM; it is deterministic. Use the button/list flows or switch the profiles to a real provider.
Message accepted but nothing happens (webhook path)	The worker is not running or cannot reach Redis. `docker compose logs worker`. `/dev/simulate` bypasses the queue, webhooks do not.
Hospital not found for channel	`channel_id` must match one of `DEMO_HOSPITAL_CHANNEL_IDS` (default `console`).
Schema errors after pulling new code	Run `docker compose up --build` so `migrate` applies new revisions; in the worst case `down -v` to recreate the database.
Line-ending errors in scripts on Windows (`\r: not found`)	Check out with LF endings (`git config core.autocrlf input`) – notably `services/staff-ui/docker/40-runtime-config.sh`.
Slow builds	Allocate more CPU/RAM to Docker Desktop; keep source on the WSL file system if using WSL.

14. Security notes for local use

- Default credentials (`hcwave/hcwave` , `dev-internal-token` , `change-me-…` secret key) are for laptops only. Postgres and Redis are published on all host interfaces; on untrusted networks bind them to loopback by writing `"127.0.0.1:5432:5432"` .
- Staff authentication is off and the internal API token is public knowledge; do not expose ports 8000/8001/5173 through tunnels except briefly for webhook testing.
- Use only synthetic data. The platform is designed to hold patient data in production (DPDP considerations: India hosting, consent, audit logging, PII-masked logs); the local stack provides none of the network or identity protections of the Azure deployment.
- Application containers run as non-root (UID 10001; nginx UID 101) and do not read a .env file inside the container.
- Row-level security separates hospitals even locally; connect as the `hcwave` role, not a superuser, if you want to test it (superusers bypass RLS).

15. Appendix – environment variable reference

Complete, commented template: `.env.example`. Grouped summary of what the local stack uses:

Variable	Local default	Notes
APP_ENV	dev	Selects `config/environments/<env>.yaml`.
APP_SECRET_KEY	change-me…	HMAC key for cookies and OTP hashing. Change even locally if you share the machine.
PUBLIC_BASE_URL	http://localhost:8000	Used to build links (e.g. report portal URLs).
REPORTS_BASE_URL	(empty)	Public portal host in production.
STAFF_UI_ORIGIN	http://localhost:5173	CORS allow-origin.
LOG_LEVEL	INFO	DEBUG for troubleshooting.
DATABASE_URL	(Compose override)	`postgresql+asyncpg://hcwave:hcwave@postgres:5432/hcwave` inside Compose; `localhost` if running the API outside Docker.
DATABASE_AUTH_MODE / DATABASE_SSL_MODE	password / disable	azure_ad / require in Azure.
MESSAGING_PROVIDER	console	acs_whatsapp | meta_cloud | console.
DEMO_HOSPITAL_CHANNEL_IDS	console	Channel IDs routed to the demo hospital.
QUEUE_PROVIDER / REDIS_URL	redis (Compose) / redis://redis:6379/0	.env template says in_memory; Compose overrides.
LLM_*_PROVIDER / LLM_*_MODEL	fake / gpt-5-mini, gpt-5, text-embedding-3-large	Four profiles: intent, reply, tagger, embedding.
KNOWLEDGE_PROVIDER	keyword	azure_ai_search | pgvector | keyword.
LANGUAGE_PROVIDER / SAFETY_PROVIDER	heuristic / none	Offline detection, no content safety.
STORAGE_PROVIDER / LOCAL_STORAGE_ROOT	local / /data/blobs	Blob volume in Compose.
EMAIL_PROVIDER / SMS_PROVIDER	console / console	Recorded, not sent; logs show recipient and purpose.
DASHBOARD_PROVIDER	builtin	Built-in KPI tiles.
HIS_PROVIDER	csv	Drop-folder CSV connector.
AUTH_PROVIDER	disabled	entra_id in cloud.
INTERNAL_AUTH_MODE / INTERNAL_API_TOKEN	shared_secret / dev-internal-token	Protects port 8001 locally.
SCHEDULER_ENGINE / FEEDBACK_ENGINE	none / jobs	No automatic scheduling locally.
TELEMETRY_EXPORTER	none	console prints OpenTelemetry output.

DEMO_PR_EMAIL, DEMO_PR_PHONE, DEMO_UNIT_HEAD_EMAIL, DEMO_UNIT_HEAD_PHONE	example values	Alert recipients for the demo tenant; with console providers nothing is actually sent.
staff-ui container only: AUTH_MODE (none|msal), API_BASE_URL, CSP_CONNECT_SRC, CSP_FRAME_SRC, ENTRA_CLIENT_ID, ENTRA_TENANT_ID, STAFF_API_SCOPE, POLL_SECONDS.		

Command cheat sheet

```
cp .env.example .env
docker compose up --build -d          # start everything
docker compose ps                     # all up? migrate = Exited (0)
curl localhost:8000/readyz            # database reachable
curl -XPOST localhost:8000/dev/simulate -H "content-type: application/json" -d "{\"text\":\"hi\"}"
docker compose logs -f api worker     # watch processing
docker compose down                   # stop (keep data)
docker compose down -v                # stop and wipe data
```